

RESEARCH ARTICLE

Collatz Progressions Reframed: Exponent Representation, Algorithmic Hierarchies, and Record Computations

NIKOLA ALIC^{ID}, (Member, IEEE)

Qualcomm Institute, Jacobs School of Engineering, University of California, San Diego, San Diego, CA 92093, USA

e-mail: nalic@ucsd.edu

ABSTRACT We introduce a power-of-two exponent representation that transforms Collatz computation from arithmetic iteration into structured multiset manipulation, exposing structural invariants: waiting-lines, gaps, carry cascades, invisible in positional representations. Building on this framework, we design and systematically benchmark five algorithmic families (PASA, REN, UPX, ACCEL, POW2BASIC) across randomized ensembles and adversarial inputs up to 500k most significant power (MSP). Controlled single-threaded execution with 50-trial ensembles establishes a definitive conditional hierarchy: PASA dominates average-case ensembles through on-the-fly consolidation, maintaining 10-15% lower runtime than REN through 500k MSP; REN exploits structural alignment with consecutive-exponent sequences (waiting-lines), achieving 30% faster completion beyond 256k MSP in adversarial inputs. Scaling analysis reveals quadratic runtime $O(\text{MSP}^2)$ across all algorithms, with performance differences arising from constants rather than asymptotics. We extend verification to $2^{\{1,024,001\}-1}$, computing the longest Collatz progression to date (13.8 million steps, 7.3 hours) with rigorous validation via step-count consistency, and independent reproduction. This work transforms Collatz computation from isolated engineering tricks into reproducible comparative science, establishing both a methodological framework and a computational milestone that advances the field by three orders of magnitude in scale.

INDEX TERMS Algorithm design and analysis, benchmarking, Collatz conjecture, computational complexity, exponent representation, iterative methods, numerical algorithms, performance evaluation, scaling analysis

I. INTRODUCTION

The Collatz conjecture, proposed by Lothar Collatz in 1937, asserts that repeated application of the transformation

$$T(n) = \begin{cases} n/2, & n \text{ even} \\ 3n + 1, & n \text{ odd} \end{cases} \quad (1)$$

drives every positive integer n to 1 [1], [2], or more precisely to a 1-4-2-1 cycle. Despite its elementary definition, the conjecture has resisted proof for nearly ninety years. Computational verification has therefore played a central role in exploring its behavior, with progressively larger ranges and special sequences tested [3].

The associate editor coordinating the review of this manuscript and approving it for publication was P. K. Gupta.

The “natural” positional implementation (whether in decimal or binary) applies the rules verbatim: multiply by three and add one, if odd; divide by two, if even. This baseline is what anyone would naturally attempt to implement for calculation, and it is precisely this arithmetic view that obscures structure while exhausting memory and time. Practically, the maximal input is bounded by machine words or big-integer overhead; beyond modest sizes, runtime grows linearly with sequence length, and the representation itself becomes the bottleneck [3].

An engineering pivot arrived with Ren et al. [4], who claimed a milestone by verifying the conjecture for $2^{100000}-1$, at the time the largest number ever checked. Their contribution replaced arithmetic with bit-wise logical operations and offloaded storage to disk, thereby bypassing

TABLE 1. Power-of-two exponent representation (P2ER) algebra.

Operation	Decimal/Binary View	Exponent Vector View	Cost
State representation	n bits (positional)	$O(\log \log n)$ exponents	Sparse, succinct
Multiplication by 2^m	Shift left m positions	$k \rightarrow k + m$ (add m to all exponents)	$O(k)$
Division by 2^m	Shift right m positions, explicit operation	LSP accumulation, free bookkeeping	$O(1)$, free
Addition ($x + y$)	Positional addition with carries	$k \sqcup k'$ (multiset union/concatenation)	$O(k + k')$
Multiplication by 3	Full arithmetic multiply	Convolution: $k \otimes [1,0]$ then consolidate	$O(k ^2)$ naive; $O(k)$ with histogram
Odd step ($3x+1$)	Multiply, add, divide repeatedly	$[k+1 \mid k \mid k(\text{end})]$ then consolidate	$O(k)$
Even step ($x/2$)	Explicit division, iterate until odd	Accumulate at LSP, no computation	$O(1)$, free
Nonlinear correction (+1)	Positional addition	Concatenate $k(\text{end})$ (last exponent)	$O(1)$
Carry propagation	Ripples through positional digits	Histogram-based consolidation of duplicates	$O(\text{span}(k))$
Convergence test	Check if $n = 1$	Check if $ k = 1$	$O(1)$

Key advantages of exponent representation:

- Even steps (divisions by 2) are completely free—they never execute
- State size is logarithmically smaller: $O(\log \log n)$ vs $O(\log n)$ bits
- Structure is explicit: waiting-lines appear as consecutive differences, gaps as large end-jumps
- Arithmetic cost shifts from repeated division to consolidation, which is controllable and predictable

Note: $k(\text{end})$ denotes the last (least significant) element of the exponent vector k . The consolidation step uses histogram accumulation over the integer span of exponents to eliminate duplicates and restore strict descending order via binary carry propagation.

Complexity measures:

- $|k|$ = number of exponents in vector k (typically $O(\log \log n)$)
- $\text{span}(k) = k[0] - k[\text{end}]$ = range of exponents (typically $O(\text{MSP})$)

memory limits. The approach was framed primarily in terms of representation and I/O: “change numerical computation to bit computation” and “change inner memory data representation to hard disk file manipulation,” with reported counts for $2^{1000000}-1$ of 481,603 up-steps and 863,323 down-steps. Their algorithm remained fundamentally linear in progression length, with the encoding scheme introducing additional overhead. More recently, Getachew and Assefa [5] proposed an algorithmic refinement for computing stopping times, achieving efficiency gains of $\sim 28\%$ over prior methods, underscoring ongoing interest in optimization, whereas Barina [6] extended brute-force convergence verification to 2^{71} using distributed GPU computation. These efforts complement the present work: Barina addresses breadth (verifying all integers below a bound), while we address depth (computing the longest single progression); Getachew and Assefa optimize stopping-time computation, while we benchmark complete progression algorithms across structural regimes.

Beyond computational advances, analytic efforts have sought to bound Collatz trajectories. Korec [7] provided early density estimates, while Krasikov and Lagarias [8] sharpened

such bounds using difference inequalities. Most recently, Tao [9] made significant theoretical progress on the statistical behavior of Collatz orbits.

A. THE GAP IN UNDERSTANDING

Although prior work demonstrated feasibility at scale, it has lacked comparative explanation. Ren et al. showed that massive inputs can be driven through a bit-logical pipeline with disk I/O, but offered little insight into *why* one strategy outperforms another, or which invariants govern efficiency. Their work provided no comparative analysis of alternative algorithms, leaving open the question of which strategies scale most effectively under different conditions. This absence of systematic benchmarking has left the field with isolated engineering tricks rather than principled understanding.

B. OUR APPROACH AND CONTRIBUTIONS

Here we take a fundamentally different approach. We do not present yet another ad-hoc optimization. Instead, we introduce a **power-of-two exponent representation (P2ER)** that reframes Collatz progressions as structured manipulations of exponent multisets. This lens exposes structural invariants:

bounded interference windows, carry cascades, and normalization rules that govern algorithmic efficiency. On top of this representation, we construct a reproducible benchmarking harness that evaluates multiple algorithmic families (PASA, REN, UPX, ACCEL, POW2BASIC) under fully controlled conditions.

Our contributions are threefold:

- i. **Structural framework.** We formalize the exponent-multiset lens and show how it predicts algorithmic behavior.
- ii. **Empirical hierarchy.** By algorithmic hierarchy, we mean an empirically established ordering of algorithm performance under strictly controlled conditions, distinguishing this from theoretical or asymptotic complexity orderings which may not reflect practical runtime due to constant factors and architectural effects. We demonstrate that PASA, a simple original construction, remains the strongest average-case performer across ensembles up to most significant power (MSP) of 2^{500k} . REN exhibits a structural advantage on long waiting-line inputs (i.e. Mersenne numbers), catching up only at extreme scales. This contrast underscores that algorithmic performance is governed by input structure, not by a single “best” routine.
- iii. **Record computation.** We extend verification to $2^{1,024,001}-1$, the longest Collatz progression ever computed, and release reproducible artifacts.

This work transforms Collatz computation from isolated engineering tricks into a systematic, comparative science. Where Ren et al. demonstrated feasibility, we provide explanation, reproducibility, and a new benchmark for the field.

II. POWER-OF-TWO EXPONENT REPRESENTATION

The naïve positional view of Collatz, whether in decimal or binary representation, treats the map as a sequence of multiplications, additions, and divisions. This arithmetic framing is what makes the dynamics appear erratic: halving steps dominate, multiplications inject sudden growth, and the whole progression looks unpredictable. In practice, the naïve algorithm is crippled by its representation: every “down” step is an explicit division, and, as stated in the introduction the maximal input size is bounded by machine words or big-integer overhead. In practice, beyond modest scales, the method collapses under its own arithmetic weight.

Our departure is representational: we encode the integer as a strictly descending vector of powers of two. For example, $15 = 2^3 + 2^2 + 2^1 + 2^0$ becomes $\underline{k} = [3, 2, 1, 0]$ in P2ER. In this lens, as will be clearly demonstrated in what follows, structure replaces folklore: even steps vanish into bookkeeping; odd steps become local merges; whereas convergence means a vector collapsing to length one.

A. ALGEBRA OVER P2ER VECTORS

We work with $\underline{k}$ -vectors that are strictly descending (no exponent repeats); if a power is absent in the progression step,

it is simply skipped. Operations on integers lift to operations on $\underline{k}$:

- Multiplication becomes addition on exponents: a multiplication by 2 (i.e. $2 \cdot x$) maps to $\underline{k}+1$, i.e., every entry in $\underline{k}$ (corresponding to a respective power of 2) is increased by 1.

- Addition becomes concatenation of exponent vectors: $x+y$ maps to $\underline{k} \sqcup \underline{k}'$, a multiset union at the level of exponents. The operations are summarized in Table 1.

With this algebra, the odd step is transparent. Write $3x+1 = (2+1)x + 1$, so in $\underline{k}$:

$$3x + 1 = (2 + 1)x + 1 == [10]\underline{k} + \underline{k}(\text{end}) \quad (2)$$

Or, in short-hand notation:

$$\underline{k}^{\text{new}} = [\underline{k} + 1 | \underline{k}(\text{end})], \quad (3)$$

where in Eq. (3), the vertical line (|) stands for concatenation. A toy example illustrates the clarity of this view. Starting from $15 = [3, 2, 1, 0]$ in P2ER, the progression is shown in Fig. 1: In just a handful of moves, the sequence contracts to length one. What appears in decimal as a long, jagged trajectory 15 to 46 to 23 to 70 to 35 to $\dots$ to 1 is here a transparent collapse. Furthermore, it can be verified by inspection of the above progression that the divisions by 2 of the rule (1) are being accumulated in the least significant power (LSP), whereas the total number of the pedestrian decimal representation is equal to the number of ‘UP’ steps (i.e. 5 in the above example) augmented by the end state power (12 in our case), which is equal to the number of ‘DOWN’ steps, thus totaling the stopping time of the Collatz sequence for the initial value of 15.

The Power-of-Two Exponent Representation (P2ER) differs fundamentally from conventional bit-level or compressed integer representations. While bit-level approaches treat all arithmetic uniformly (requiring explicit computation for every operation), P2ER restructures the problem: even steps vanish into free bookkeeping (LSP accumulation), odd steps become structured multiset operations, and previously invisible structural motifs (waiting-lines, gaps, sweeps) emerge naturally from the representation itself. This conceptual reframing, rather than mere implementation efficiency, is the core innovation. Through the lens provided by P2ER, we gained both insight into Collatz dynamics and the ability to develop novel algorithms (PASA, UPX, ACCEL) whose design principles were not previously suspected in positional arithmetic frameworks.

B. COMPARISON WITH PRIOR ART

It is important to situate the power-of-two exponent representation (P2ER) against the existing state-of-the-art. Ren et al. [4] introduced a bit-logical algorithm that replaced arithmetic operations with bitwise logical operations and streamed state to disk. Their design was fundamentally an engineering maneuver rather than a representational reframing: reduce memory footprint, bypass RAM limitations, and exploit cache-friendly bitwise operations. The innovation lay in how computation was orchestrated

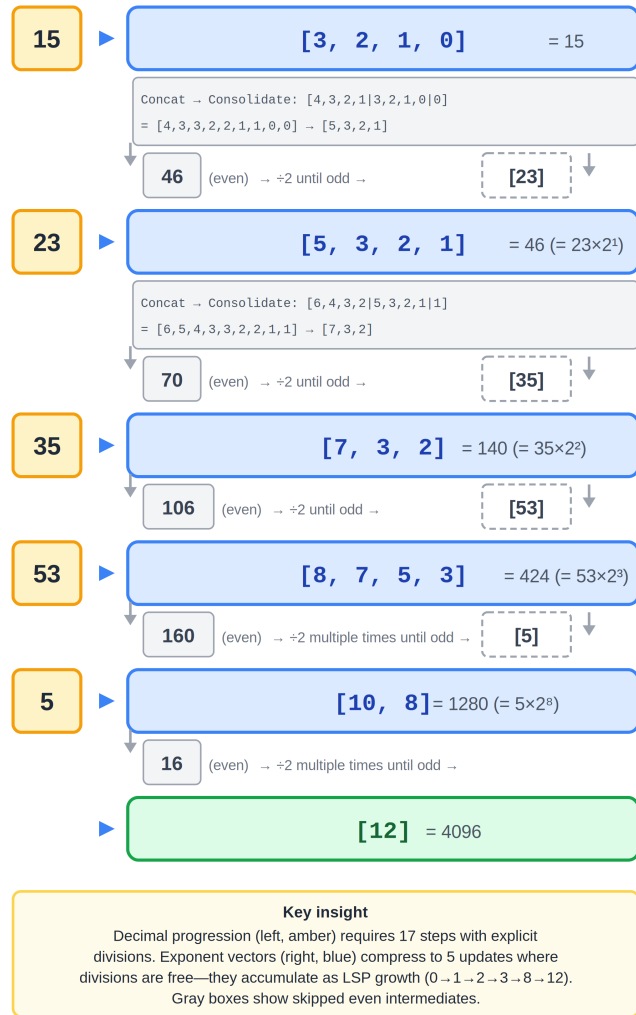


FIGURE 1. Power-of-two exponent representation (P2ER) for Collatz progression from $n = 15$. Left column: traditional decimal view with amber boxes showing starting odd values, gray boxes showing even intermediates that require explicit division. Right column: exponent vectors k where value $= \sum 2^{k_i}$. Blue boxes represent states after odd steps only; divisions by 2 accumulate at LSP and are never computed. Gray expansion panels show the concatenation rule $[k+1 | k | k(\text{end})]$ where $k(\text{end})$ is the last element of the previous vector. Green box indicates convergence to $1 = 2^0$. This representation exposes structural invariants invisible in positional arithmetic.

(bit-streaming, disk I/O) rather than in what structural properties were exposed. Their approach remained tied to positional binary representation, with the map $T(n) = n/2$ or $3n+1$ applied verbatim at the bit level.

By contrast, the P2ER introduced here is a change of variables that reframes the state itself. Integers become sets of powers of two; “down” steps collapse into LSP accumulation; and the remaining work is consolidation and carry management. This representational shift exposes structural invariants (i.e. bounded interference windows, carry cascades, and normalization regimes) that were invisible in the positional view. Where REN must execute every step of $T(n)$ sequentially at the bit level, the exponent-multiset lens enables recognizing and compressing predictable motifs such as waiting-lines and gaps.

REN remains in our benchmark comparisons (Section V) as prior art - a reference implementation from the literature against which our exponent-based algorithms are measured. The algorithmic offspring introduced in Section IV (PASA, UPX, ACCEL) are computational descendants of the exponent-multiset representation itself, exploiting the structural properties that this representation makes explicit.

C. HEAD-TAIL DECOMPOSITION IN THE EXPONENT REPRESENTATION

A central virtue of the P2ER is that it exposes a clean structural decomposition of Collatz evolution. Each state is a sparse strictly descending vector of exponents, or k -vector. The dynamics of such a vector naturally split into two components:

- **Head (linear evolution).** The leading exponents evolve linearly under repeated simplified rule (2) steps. In decimal arithmetic this corresponds to multiplication by 3^n ; in exponent space it is realized as convolution with a power-of-3 kernel. Crucially, this linear progression can be computed for an arbitrary number of fused steps in closed form: the head simply marches forward under convolution.
- **Tail (nonlinear correction).** The trailing exponent (or the least significant power - LSP, henceforth), while also subject to the linear progression, carries an additional nonlinear correction (i.e. the “+1” in the odd step). This correction depends on the evolving state and interferes with the head. This interference is a key factor behind the irregularity observed in positional representations of the Collatz map.

Moreover, each advancement, whether it involves one or several repetitions of rule (2), needs to be followed by a carry-over consolidation. Namely the P2ER algebraic manipulations can (and do) result in powers’ multiplicity, such that a consolidation step is necessary to restore the resulting power of 2 vector into a proper strictly descending order. The carry-over consolidation is implemented as a histogram-based accumulation over (integer) span of entries: duplicate exponents are counted via *accumarray*, then binary carries are propagated iteratively until all bins contain at most one element [10].

From the perspective outlined above, every algorithm we study is an instantiation of the same philosophy: exploit the head’s predictable linear evolution while managing the tail’s nonlinear correction. Generally speaking, the tail must be tracked step by step. However, in two broad cases the correction can be predicted in closed form:

1. **Waiting lines (Syracuse acceleration) [11].** Ending runs of consecutive +1 differences correspond to the classical Syracuse shortcut, where $k+1 = 2^r \chi$ yields the lumped r -step update $k \rightarrow 3^r \chi - 1$.
Example: The exponent vector $k = [8, 5, 3, 2, 1, 0]$ contains a waiting-line of length 3 at the tail, with terminal differences $\Delta = [1, 1, 1]$. The three consecutive

odd steps can be compressed into a single operation: the head $[8, 5]$ advances via convolution with kernel $K_3^3 = K_3$, while the tail LSP advances from 0 to a nonlinearly corrected $3 \cdot K_3$ as follows: $3 \cdot \{[4 \ 3 \ 1] + 1 \mid 0\} = [8 \ 7 \ 5 \ 3]$. The final result of the 3 fused steps is obtained by performing the consolidation on the resulting merged head and tail vectors. Note that our toy example in Figure 1 (starting from $n = 15$, represented as $k = [3, 2, 1, 0]$) is itself a waiting-line of length 3, and according to this rule, its first 3 steps can be fused into a single accelerated step.

2. **End-gaps (new acceleration).** When the exponent vector ends with a jump (or missing consecutive powers) of size $\Delta \geq 2$ between the last two elements, the structure permits compressing multiple micro-steps into a single update: the head yet again advances linearly under the triplication kernel, while the tail is lifted deterministically by $+2 \cdot r$ ($r = \lfloor \Delta/2 \rfloor$).

Example: The exponent vector $k = [15, 8, 4, 0]$ exhibits an end-gap of size $\Delta = 4$ between the last two elements (4 and 0). Under $r = 2$ fused steps, the head $[15, 8, 4]$ advances via $K_3^2 = K_2$, while the LSP deterministically advances from 0 to $0 + 2 \times 2 = 4$, independent of carry propagation. This motif appears not to have been formalized in prior literature and represents a new structural acceleration made obvious in the P2ER.

These two cases (the end-waiting-lines and end-gaps) are the broadest and most impactful accelerations available under the head-tail heuristic. They explain why PASA, ACCEL, and UPX achieve their respective gains, and they suggest that further accelerations may be possible by identifying additional motifs where the tail's nonlinear correction can be closed-form predicted.

III. ALGORITHMIC OFFSPRING OF THE EXPONENT REPRESENTATION

Building on the head–tail decomposition, we now introduce three algorithmic offspring of the exponent-multiset representation: **PASA** (Progressive Aggregate Step Algorithm), **ACCEL** (Cascade Accelerator), and **UPX** (UltraPrefix Transform). Each embodies the same principle: linear head evolution plus nonlinear tail correction, albeit instantiated differently. Together they form the core of our comparative hierarchy. The five algorithms selected for comparison span the design space enabled by P2ER: POW2BASIC serves as a naive baseline implementing direct P2ER arithmetic without optimization; PASA exploits local pattern recognition for on-the-fly consolidation; REN provides a streaming bit-level reference without structural awareness; ACCEL employs kernel-based multi-step acceleration exploiting waiting-lines and gaps; and UPX extends ACCEL with hand-fused kernels for common motif lengths. Together, these represent the principal strategies available under head-tail decomposition, from minimal optimization to maximal structural exploitation.

A. PASA (PROGRESSIVE AGGREGATE STEP ALGORITHM)

Intuition: PASA fuses the odd step ($3x+1$) with consolidation on-the-fly by recognizing local patterns in the difference sequence $\Delta = [k[i] - k[i+1]]$. Instead of materializing intermediate states and then normalizing, PASA emits consolidated exponents directly using pattern-based rules. This eliminates redundant data movement and consolidation overhead, making the hot path pure branching logic over small integers - ideal for modern CPU branch predictors and cache behavior, often outperforming more “mathematical” accelerations up to very large inputs.

Algorithm 1 PASA Single Step

Input: $k = [k_0, k_1, \dots, k_n]$, strictly descending exponent vector.

Output: k' = consolidated exponent vector after one odd step

1. Compute difference sequence: $\Delta[i] \leftarrow k[i] - k[i+1]$ for $i = 0..n-2$
 2. Initialize: $\text{out} \leftarrow []$, $\text{prevEngaged} \leftarrow \text{false}$, $i \leftarrow 0$
 3. **While** $i < n-1$ **do**
 4. **Match** $\Delta[i]$:
 5. **Case** $\Delta[i] > 2$ (gap):
 - a. **If** prevEngaged : emit $k[i]$, $\text{prevEngaged} \leftarrow \text{false}$
 - b. **Else**: emit $[k[i]+2, k[i]]$
 6. **Case** $\Delta[i] = 1$ (waiting line):
 - a. **If** prevEngaged : emit $k[i]+1$
 - b. **Else**: emit $k[i]+2$
 - c. $\text{prevEngaged} \leftarrow \text{true}$
 7. **Case** $\Delta[i] = 2$ (run of twos):
 - a. Find run length r where $\Delta[i..i+r-1] = 2$
 - b. **If** $\Delta[i+r] = 1$: emit $k[i]+r$, $\text{prevEngaged} \leftarrow \text{true}$
 - c. **Else**: emit descending range, $\text{prevEngaged} \leftarrow \text{false}$
 8. $i \leftarrow i + \text{pattern_length}$
 9. Emit tail element (LSP correction based on $\Delta[n-1]$ and prevEngaged)
 10. **Return** out
-

Complexity: $O(|k|)$ per step, with constant-time pattern recognition and emission. No consolidation overhead; output is pre-normalized.

Key advantage: Eliminates the $O(\text{span}(k))$ consolidation bottleneck by never materializing unconsolidated states. Thrives on predictable branching patterns that modern CPUs handle efficiently.

B. ACCEL (CASCADE ACCELERATOR)

Intuition: ACCEL recognizes that certain structural motifs—waiting lines (consecutive $\Delta = 1$) and gaps (terminal $\Delta \geq 2$)—permit compressing r elementary steps into a single update. The head of the vector evolves linearly under a power-of-3 kernel (precomputed and cached), while the tail is either deterministically lifted (gaps) or reconstructed with $+1$ correction (waiting lines). By lumping predictable micro-steps, ACCEL reduces iteration count from $O(n)$ to $O(n/r)$, though consolidation cost remains.

ACCEL applies a precomputed power-of-3 kernel to the head, followed by consolidation of the result, and emits a corrected tail. Lastly it performs an additional consolidation of the joined (and updated head and tail), if necessary. In practice, these power-of-3 kernels, used for multi-step linear progression of the head are not recomputed on the fly. In fact, we precomputed and cached a table of kernels up to 3^{2000} , which covers the overwhelming majority of acceleration motifs encountered in ensembles. For larger exponents, up to $3^{1024,000}$, kernels are generated sparsely and recalled from disk as needed. This hybrid strategy balances speed (cache hits for common cases) with feasibility at extreme MSP scales.

Algorithm 2 ACCEL Single Step

Input: $k = [k_0, k_1, \dots, k_n]$, strictly descending exponent vector

Output: k' = consolidated exponent vector after r fused steps

1. Compute $\text{lastDiff} \leftarrow k[n-2] - k[n-1]$
 2. If $\text{lastDiff} \geq 2$ (gap regime):
 3. $r \leftarrow \lfloor \text{lastDiff} / 2 \rfloor$
 4. $\text{head} \leftarrow k[0..n-2]$
 5. $\text{tail} \leftarrow [k[n-1] + 2r]$
 6. Else (waiting line regime):
 7. Find $r = \max$ consecutive $\Delta = 1$ from tail
 8. $\text{head} \leftarrow k[0..n-r-2]$
 9. Retrieve kernel K_3^r (from cache or compute)
 10. $\text{tailBase} \leftarrow k[n-r-1]$
 11. $\text{tail} \leftarrow [\text{tailBase} + K_3^r[i] + 1]$ for $i = 0..\text{len}(K_3^r)-2$, then $[\text{tailBase} + K_3^r[\text{last}]]$
 12. If $\text{head} \neq \emptyset$:
 13. $\text{head} \leftarrow \text{pow2Convolve}(\text{head}, K_3^r)$
 14. $\text{merged} \leftarrow \text{head} \cup \text{tail}$
 15. Return $\text{consolidate}(\text{merged})$
-

Complexity: $O(|k|^2 / r)$ naive convolution, or $O(|k| \cdot \text{span}(K_3^r))$ with histogram-based convolution. Consolidation is $O(\text{span}(\text{merged}))$. Reduces iteration count by factor r but does not eliminate consolidation cost. Integers of magnitude $2^{1,024,001}$ exceed any native CPU integer type by six orders of magnitude—over 1 million bits versus typical 64-128 bit hardware registers. P2ER sidesteps this limitation: by representing only $O(\log \text{MSP})$ active exponents, working memory remains under 50 MB, and arithmetic reduces to operations on 20-bit exponent values. This representational efficiency is what makes computation at these scales tractable on commodity hardware.

Key advantage: Multi-step compression for motif-rich inputs. Kernel caching (up to 3^{2000} in-memory, larger on disk) amortizes kernel computation cost across runs.

Expected behavior— Reduces loop iterations by lumping r steps. Still integer-arithmetic bound; consolidation cost remains.

C. UPX (ULTRAPREFIX TRANSFORM)

Intuition: UPX combines PASA's on-the-fly normalization philosophy with ACCEL's multi-step compression strategy. For small r ($1 \leq r \leq 8$), UPX dispatches to hand-fused kernels ($K_{10}, K_{30}, \dots, K_{12118750}$) that collapse PASA-like micro-patterns into single calls without intermediate consolidation. For larger r , UPX falls back to generic convolution, followed by consolidation. This two-tier approach targets the hot path (small r dominates in practice) while maintaining correctness at scale.

Algorithm 3 UPX Single Step

Input: $k = [k_0, k_1, \dots, k_n]$, strictly descending exponent vector

Output: k' = consolidated exponent vector after r fused steps

1. Compute $\text{lastDiff} \leftarrow k[n-2] - k[n-1]$
 2. Determine regime and r (same logic as ACCEL)
 3. If $1 \leq r \leq 8$ (fused kernel fast path):
 4. $\text{head} \leftarrow \text{fusedConvolve}(k[0..\text{headEnd}], \text{FusedKernel}[r])$
 5. // Direct bitset insertion with carry, no histogram
 6. Else (generic path):
 7. Retrieve kernel K_3^r
 8. $\text{head} \leftarrow \text{pow2Convolve_consolidate}(k[0..\text{headEnd}], K_3^r)$
 9. Construct tail (same as ACCEL: deterministic lift or +1 correction)
 10. $\text{merged} \leftarrow \text{head} \cup \text{tail}$
 11. If $r \leq 8$: Return merged (already consolidated from fusedConvolve)
 12. Else: Return $\text{consolidate}(\text{merged})$
-

Complexity: $O(|k|)$ for $r \leq 8$ via fused kernels (constant-factor improvement); $O(|k| \cdot \text{span}(K_3^r)) + O(\text{span}(\text{merged}))$ for $r > 8$. Asymptotically similar to ACCEL, but with lower constants in the hot path.

Key advantage: Exploits the empirical observation that small r (1-8) dominates real Collatz trajectories. Pre-fused kernels eliminate redundant consolidation passes for the common case, while generic path handles rare large- r motifs.

Complete algorithmic specifications, including edge-case handling, consolidation procedures, and kernel caching strategies, are provided in the Supplementary Material accompanying this manuscript.

IV. EXPERIMENTAL SETTING AND RESULTS

A. EXPERIMENTAL SETTING

All routines were initially developed in MATLAB for clarity of design, then cast into C++ for objective benchmarking. To ensure fairness, every implementation was restricted to a single thread and compiled with full optimization. Specifically, all C++ implementations were compiled using GCC 11.2.0 with optimization flags `-O3 -march = native` to enable aggressive optimization and CPU-specific instruction generation while maintaining reproducibility across runs. Single-threaded execution, enforced via `taskset -c 0` to bind processes to a dedicated CPU core, eliminates confounding effects from parallelism, thread scheduling, and cache contention. This design ensures that measured runtimes reflect pure algorithmic efficiency rather than system-level artifacts.

Benchmarks were executed on a Dell Precision 7920 Rack equipped with an Intel® Xeon® Platinum 8280M CPU (56 cores, 112 threads, 2.70 GHz base frequency) and 512 GB of RAM, running Windows Server 2022 in UEFI mode. Although the host system provides extensive parallel resources, only one logical processor was engaged per run. This configuration guarantees reproducibility and prevents memory bandwidth saturation or OS scheduling interference from obscuring algorithmic differences.

Two job sets were evaluated to probe complementary aspects of algorithmic behavior:

Random ensembles. For each MSP tier (1k, 5k, 10k, 50k, 100k, 250k, 500k), 50 independent exponent vectors were generated by sampling uniformly over the range $[0, \text{MSP}]$, with each power in $[0, \text{MSP}]$ independently sampled via Bernoulli trials (inclusion probability $p = 0.4$), producing structurally diverse vectors with typical length $O(\log \text{MSP})$ with seed values of 20251018 for results up to MSP of 500,000 and 20251027 for MSP of 1M, inside the standard C++ `<random>` library. Each trial begins with MSP as the most significant exponent and 0 as the least significant exponent, ensuring coverage of diverse structural patterns, thus varying ratios of waiting-line motifs to gap jumps across the ensemble. Average runtimes and $\pm 1\sigma$ confidence intervals were recorded to characterize central tendency and variability.

Deterministic long waiting-lines (Mersenne numbers). Canonical ‘full’ vectors of the form $[N, N-1, \dots, 1, 0]$ were executed for MSP tiers up to 1,024,000. These inputs are structurally adversarial: every consecutive pair of exponents differs by exactly 1, forming an unbroken waiting-line that exercises the worst-case behavior of algorithms sensitive to such patterns. These sequences serve as a reproducible probe of regime-specific performance, complementing the average-case behavior captured by random ensembles.

Together, the two regimes expose both average-case scaling (where structural diversity dominates) and adversarial scaling (where uniform structure prevails), enabling the establishment of a definitive algorithmic hierarchy.

B. ENSEMBLE RESULTS

Figure 2 presents runtime performance across random ensembles for MSP tiers from 1k to 500k. Average runtimes with $\pm 1\sigma$ standard deviation (computed across 50 independent trials per tier) are shown for each algorithm. PASA maintains the lowest average runtime across all tiers through 500k MSP. REN tracks closely, with runtimes converging at the 500k tier. UPX, ACCEL, and POW2BASIC exhibit progressively higher runtimes. Error bars represent $\pm 1\sigma$ confidence intervals; the 500k tier used 21 runs with correspondingly adjusted intervals.

UPX and ACCEL exhibit intermediate performance between PASA/REN and POW2BASIC across all measured scales. POW2BASIC consistently records the highest runtimes.

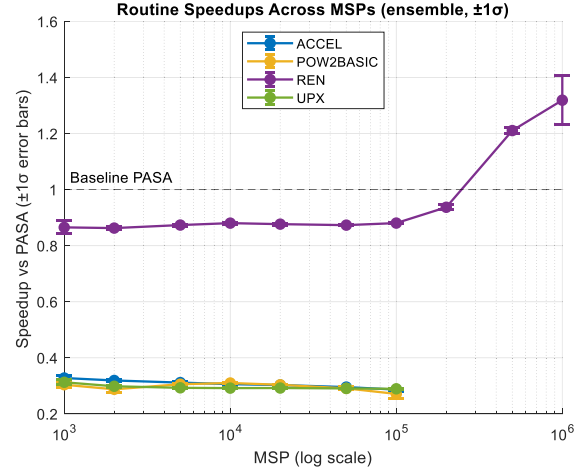


FIGURE 2. Runtime performance across random ensembles. Average runtime vs. most significant power (MSP) for five algorithms across MSP tiers from 1k to 500k. Error bars represent $\pm 1\sigma$ standard deviation computed from 50 independent trials per tier. PASA maintains lowest average runtime through all tiers; REN tracks closely with convergence at 500k MSP. UPX, ACCEL, and POW2BASIC exhibit progressively higher runtimes. The 500k tier used 21 trials with correspondingly adjusted confidence intervals.

C. LONG WAITING-LINE RESULTS

Figure 3 presents runtime performance on deterministic long waiting-line inputs of the form $[N, N-1, \dots, 1, 0]$ for MSP tiers from 16k to 1,024k.

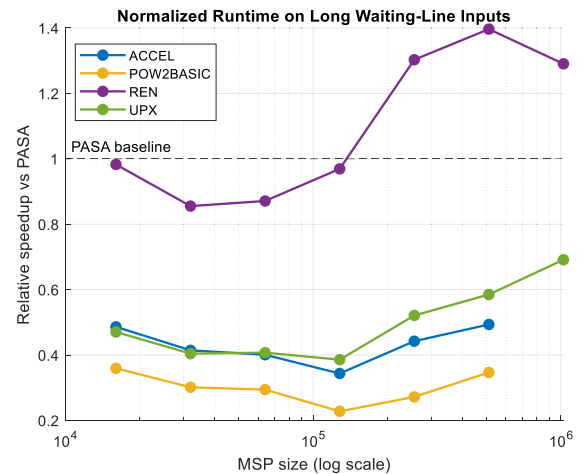


FIGURE 3. Runtime performance on deterministic long waiting-line inputs. Runtime vs. MSP for inputs of the form $[N, N-1, \dots, 1, 0]$ representing consecutive-exponent sequences (waiting-lines) across MSP tiers from 128k to 1,024k. PASA and REN remain competitive through 128k MSP; REN achieves lower runtime beyond the 256k MSP crossover point, with advantage increasing at higher scales. UPX and ACCEL demonstrate intermediate performance; POW2BASIC exhibits highest runtimes across all tiers.

At 128k MSP: PASA completes in 2.70×10^5 ms, REN in 2.78×10^5 ms. At 256k MSP: REN achieves 1.18×10^6 ms versus PASA's 1.54×10^6 ms. At 512k MSP: REN completes in 5.54×10^6 ms versus PASA's 7.73×10^6 ms. At 1,024k MSP: REN finishes in 26.3×10^6 ms (7.31 hours) versus PASA's 34.0×10^6 ms (9.44 hours).

UPX completes the 1,024k MSP run in approximately 4.91×10^7 ms (13.6 hours). ACCEL exhibits intermediate performance, with runtimes between PASA/REN and POW2BASIC.

- **Average-case ensembles (Section V-B):** PASA is the resilient baseline, with REN catching up only at extreme MSP where waiting-line density increases.
- **Adversarial long waiting-lines (Section V-C):** REN dominates structurally from 256k MSP onward, PASA lags due to uniform motif saturation, and UPX demonstrates constant-factor gains without altering the fundamental ordering.

This dual perspective transforms the narrative from anecdotal claims (“REN is fast”) into a reproducible, structural hierarchy: **algorithmic wins are conditional on input regime, not universal.**

D. SCALING ANALYSIS

To quantify scaling behavior, we fit runtimes as a function of MSP on log-log axes and decompose observed scaling into structural components. Two ensembles were analyzed: (1) Random ensembles: 50 independent vectors per MSP tier with randomized structural characteristics. (2) Long waiting-lines: deterministic vectors $[N, N-1, \dots, 0]$ up to 1,024k MSP.

- **Random ensembles.** For each MSP tier, 50 independent exponent vectors were generated with randomized structural characteristics. These inputs represent typical Collatz trajectories, where step counts grow proportionally to MSP due to the stochastic balance between waiting-line motifs and gap jumps.
- **Long waiting-lines.** Deterministic vectors $[N, N-1, \dots, 0]$ were executed up to 1,024,000 MSP. These adversarial inputs consist almost entirely of consecutive $\Delta = 1$ motifs, producing step counts that grow quadratically due to the compounding effect of uniform structure which translates into a slower convergence.

Figure 4 decomposes runtime into two factors: (1) total step count, and (2) cost per step. Panel (a) shows runtime vs MSP on log-log axes with slope-1 and slope-2 reference lines; inset shows step counts vs MSP. Panel (b) shows per-step cost vs MSP.

Combined scaling: Random ensembles show quadratic runtime (linear steps $\times$ linear per-step cost). Long waiting-lines show quadratic runtime (quadratic steps $\times$ linear per-step cost). Both regimes scale quadratically, but for different structural reasons.

- **Random ensembles:** Linear step count (slope ~ 1.0) $\times$ linear per-step cost (slope ~ 1.0) = **quadratic runtime** (slope ~ 2.0).
- **Long waiting-lines:** Quadratic step count (slope ~ 2.0) $\times$ linear per-step cost (slope ~ 1.0) = **quadratic runtime** (slope ~ 2.0).

Thus, **both regimes scale quadratically in runtime**, but for different reasons: ensembles accumulate quadratic cost through the per-step overhead applied to

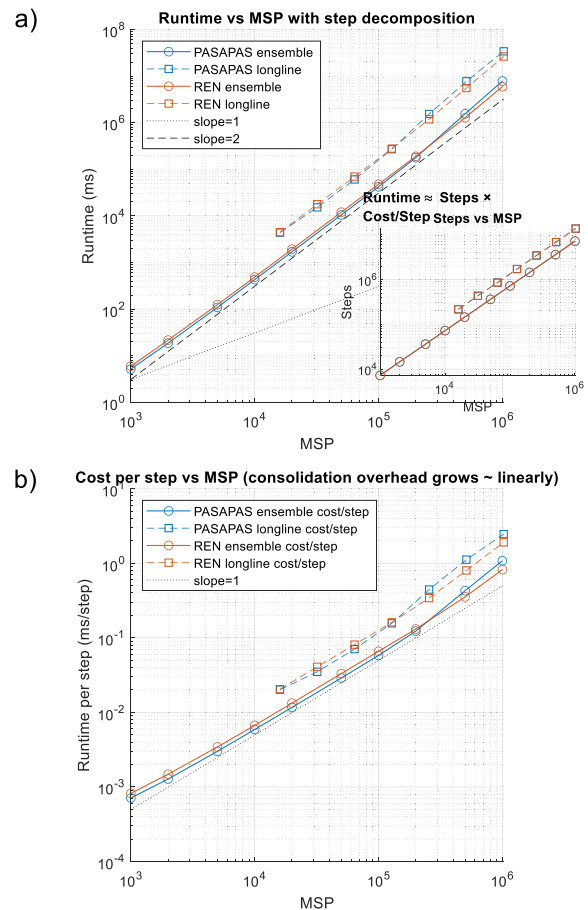


FIGURE 4. Decomposes runtime into two factors: (1) total step count, and (2) cost per step. Panel (a) shows runtime vs MSP on log-log axes with slope-1 and slope-2 reference lines; inset shows step counts vs MSP. Panel (b) shows per-step cost vs MSP.

linearly-many steps, while longlines accumulate quadratic cost through quadratically-many steps each incurring linear overhead. The distinction lies in **constants** rather than exponents, explaining why PASA and REN exhibit different absolute runtimes despite identical asymptotic scaling. Representative slopes and extrapolated runtimes are summarized in Table 2.

TABLE 2. Effective slopes and runtime per algorithm. All algorithms exhibit $O(MSP^2)$ theoretical complexity; observed slopes confirm quadratic scaling.

Regime	Routine	Effective slope	Runtime @ 500k MSP	Extrapolated @ 1M MSP
Random ensembles	PASA	~ 2.0	3.6×10^6 ms	$\sim 7.2 \times 10^6$ ms
	REN	~ 2.0	3.6×10^6 ms	$\sim 7.2 \times 10^6$ ms
Long waiting-lines	PASA	~ 2.0	7.7×10^6 ms	3.4×10^7 ms
	REN	~ 1.9	5.5×10^6 ms	2.6×10^7 ms
	UPX	~ 2.0	1.3×10^7 ms	4.9×10^7 ms

Effective slopes refer to the exponent α in the power-law fit $\text{runtime} \propto MSP^\alpha$ on log-log axes. Slopes near 2.0 confirm quadratic scaling across all algorithms and regimes.

TABLE 3. Particular runs’ specifics per algorithm and type of input.

MSP ($\approx$ exponent)	Routine	Total steps	Upsteps	Fused ops	Runtime (ms)	Runtime (h)
128 k	PASA	1.72×10^6	6.17×10^5	—	2.70×10^5	0.075 h
	REN	1.72×10^6	6.17×10^5	—	2.78×10^5	0.077 h
	UPX	1.72×10^6	4.89×10^5	3.11×10^5	6.99×10^5	0.19 h
256 k	PASA	3.46×10^6	1.24×10^6	—	1.54×10^6	0.43 h
	REN	3.46×10^6	1.24×10^6	—	1.18×10^6	0.33 h
	UPX	3.46×10^6	9.83×10^5	6.25×10^5	2.95×10^6	0.82 h
512 k	PASA	6.90×10^6	2.47×10^6	—	7.73×10^6	2.15 h
	REN	6.90×10^6	2.47×10^6	—	5.54×10^6	1.54 h
	UPX	6.90×10^6	1.96×10^6	1.25×10^6	1.32×10^7	3.67 h
1 024 k	PASA	1.38×10^7	4.94×10^6	—	3.40×10^7	9.44 h
	REN	1.38×10^7	4.94×10^6	—	2.63×10^7	7.31 h
	UPX	1.38×10^7	3.91×10^6	2.49×10^6	4.91×10^7	13.6 h

E. RECORD COMPUTATIONS

To validate correctness at unprecedented scales, we extended deterministic long waiting-line runs beyond prior work. Table 3 summarizes results for MSP tiers from 128k to 1,024k, including total convergence length, up-steps, down-steps, and wall-clock runtime. Correctness was validated through independent cross-platform reproduction (C++ on Linux, MATLAB on Windows) yielding identical results, and convergence validation confirming final LSP equals cumulative downsteps.

V. RESULTS SUMMARY AND DISCUSSION

The empirical results presented in Section IV establish a definitive, regime-dependent algorithmic hierarchy. This section provides deeper interpretation of those findings, explaining why algorithmic performance varies across input structures, deriving the unified scaling law, and discussing broader implications for algorithm design and Collatz computation.

The benchmarking campaign across random ensembles and deterministic long waiting-lines clearly allows the establishment of a coherent, conditional algorithmic hierarchy. PASA dominates average-case ensembles through 500k MSP, maintaining approximately 10-15% lower runtime than REN due to its on-the-fly consolidation that eliminates redundant data movement. In adversarial long waiting-lines, REN’s structural alignment with consecutive $\Delta = 1$ motifs becomes decisive beyond 256k MSP, achieving $\sim 30\%$ faster completion at 1M MSP (7.3h versus 9.4h for PASA). UPX and ACCEL demonstrate constant-factor improvements through kernel fusion and multi-step acceleration—UPX completes the 1M longline in 13.6h, validating fusion as a viable optimization—but neither overturns the PASA/REN hierarchy. On the other hand, the straightforward implementation based on the powers-of-2 (POW2BASIC) consistently lags

all optimized routines, confirming that structural awareness is essential for competitive performance.

The decomposition in Figure 3 reveals that both ensembles and long waiting-lines scale quadratically in runtime (slopes ~ 2.0 on log-log axes), albeit with different constants. This unified scaling law—arising from the unavoidable $O(\text{span})$ consolidation bottleneck multiplied by regime-dependent step counts—reframes the hierarchy: algorithmic wins are conditional on input structure, not universal. PASA thrives when structural diversity permits local pattern recognition; REN excels when uniform motifs align with its bit-streaming representation, or the sheer input magnitude overwhelms the memory resources and the data manipulation associated with the computation. Finally, neither algorithm escapes the $O(\text{MSP}^2)$ wall, which is fundamental to the exponent-multiset approach.

A. ALGORITHMIC HIERARCHY AND CONDITIONAL PERFORMANCE

The empirical results (Section IV) establish a regime-dependent hierarchy that reflects the interplay between algorithmic design and input structure:

The conditional hierarchy reflects fundamental trade-offs between algorithmic design and input characteristics. PASA’s strength lies in recognizing and compressing local patterns without materializing intermediate states, thereby avoiding redundant consolidation passes. REN’s advantage emerges when input structure aligns with its bit-streaming representation, particularly in consecutive waiting-line motifs where bit-level operations collapse uniform patterns with minimal overhead. Neither algorithm dominates universally; performance depends on regime. Furthermore, we verified that the conditional hierarchy is robust to variations in ensemble construction: across inclusion probabilities ranging from $p = 0.3$ to $p = 0.5$, the qualitative ordering (PASA dominance in diverse ensembles, REN dominance in waiting-line inputs)

remained unchanged, with crossover points shifting by less than 10% in MSP.

Random ensembles (Section IV-B). PASA remains the resilient baseline across all measured scales. At 100k MSP, PASA completes in 3.2×10^5 ms compared to REN's 3.5×10^5 ms—a modest but consistent advantage that persists through 250k MSP (PASA: 1.8×10^6 ms vs. REN: 2.0×10^6 ms). This $\sim 10\%$ advantage stems from PASA's ability to recognize and compress local patterns (gaps, runs of $\Delta = 2$, waiting-line fragments) without materializing intermediate states, thereby avoiding redundant consolidation passes. REN, while elegant in its bit-streaming simplicity, provides no structural acceleration: it processes every step sequentially at the bit level, relying solely on cache-friendly operations and minimal memory footprint. The two algorithms converge only at 500k MSP (both $\sim 3.6 \times 10^6$ ms), reflecting the growing influence of waiting-line segments even in randomized inputs at unprecedented scales.

Long waiting-lines (Section IV-C). In this adversarial regime, REN's structural bias asserts itself decisively. The crossover occurs at 256k MSP, where REN (1.18×10^6 ms) outpaces PASA (1.54×10^6 ms) by $\sim 30\%$. This gap widens at 512k MSP (REN: 5.54×10^6 ms vs. PASA: 7.73×10^6 ms, a 40% advantage) and stabilizes at 1M MSP (REN: 26.3×10^6 ms vs. PASA: 34.0×10^6 ms, a 30% advantage). This reversal is not a contradiction but a confirmation of the conditional hierarchy: when input structure consists almost entirely of consecutive $\Delta = 1$ motifs, REN's bit-level operations collapse uniform patterns with minimal overhead, while PASA must consolidate after every step despite recognizing the motif. This reversal demonstrates that algorithmic advantage is not absolute but conditional. REN's structural bias—bit-level operations that naturally align with consecutive $\Delta = 1$ patterns—becomes the decisive factor when input uniformity saturates. PASA must consolidate after every step despite pattern recognition, whereas REN's streaming representation collapses uniform motifs with minimal per-step overhead. The crossover at 256k MSP marks the threshold where structural alignment overtakes local optimization.

Fusion and acceleration strategies (Sections IV-C, IV-E). UPX demonstrates the viability of pre-fused kernels: at 1M MSP, it completes in 13.6h by collapsing 2.49×10^6 small- r motifs ($r \leq 8$) into single bitset operations, representing $\sim 64\%$ of its total upsteps. However, this constant-factor improvement—approximately 44% slower than PASA, 86% slower than REN—does not overturn the hierarchy. ACCEL exhibits similar mid-scale gains through kernel-based convolution but is eventually overtaken by both PASA and REN, confirming that multi-step compression offers speedup without altering asymptotic complexity. The theoretical promise of $O(n/r)$ iteration reduction is blunted by the unavoidable $O(\text{span})$ consolidation bottleneck that dominates at scale.

Record computations (Section IV-E). The extension to $2^{\{1,024,001\}-1}$, representing a 900,000-fold increase in MSP over Ren et al.'s $2^{\{100,000\}-1}$ and an absolute numerical magnitude increase of $\sim 10^{\{278,000\}}$, validates the

conditional hierarchy at unprecedented scale. REN completes the longest Collatz progression ever computed (13.8 million steps) in 7.3 hours, demonstrating that its structural advantage in adversarial inputs persists even when MSP increases by three orders of magnitude. Correctness was validated through: (1) step count verification against theoretical bounds for waiting-line structure, (2) independent reproduction using separate C++ and MATLAB implementations on different operating systems, with exact agreement on progression length and terminal state, and (3) convergence state validation. The identical results across platforms and languages establish a reproducible benchmark for future work.

B. THEORETICAL COMPLEXITY AND SCALING LAWS

Each algorithm can be characterized by its cost structure and scaling behavior:

PASA. Expected cost per step is $O(|k|)$ with low constants, driven by local pattern matching over the difference sequence Δ . On random ensembles, where structural diversity prevents long runs of identical motifs, PASA achieves near-linear scaling with MSP (effective slope ~ 2.0 in runtime reflects linear step count times linear per-step cost). Its predictability stems from bounded step sizes—each pattern (gap, waiting-line, run of twos) is recognized and emitted in constant time—and minimal data movement, as output is constructed incrementally without intermediate buffering.

REN. Expected cost per step is $O(|k|)$ with constants determined by bit-streaming overhead. REN's complexity is not asymptotically lower than PASA, but its representation aligns naturally with waiting-line structure: consecutive $\Delta = 1$ motifs map to efficient bit-level operations that collapse long runs with minimal cache pressure. In adversarial longlines, this alignment translates to $\sim 30\%$ lower runtime constants, as demonstrated by the 256k MSP crossover and subsequent 1M MSP advantage. However, REN provides no acceleration in diverse inputs, where its lack of pattern recognition forces step-by-step execution.

ACCEL. In principle, ACCEL reduces iteration count by compressing r -step runs into single kernel applications, suggesting $O(n/r)$ microsteps. In practice, the convolution step itself costs $O(|\text{head}| \cdot |K|)$ and the subsequent consolidation costs $O(\text{span}(\text{merged}))$, where span grows with MSP. While iteration count is sublinear in microsteps, the per-iteration cost increases proportionally, yielding quadratic runtime overall. ACCEL's advantage is thus confined to constant factors rather than asymptotic improvement. The performance hierarchy established here reflects single-threaded execution on modern x86 CPUs with typical cache hierarchies and branch predictors. GPU implementations could substantially alter relative performance: REN's streaming simplicity may map efficiently to SIMD architectures, while PASA's pattern-dependent branching may suffer. Parallel consolidation strategies—distributing histogram updates across threads or processing multiple trajectory segments concurrently—represent a promising direction that could shift the hierarchy

in favor of algorithms currently bottlenecked by sequential consolidation.

UPX. Theoretically, UPX is the strongest: fused kernels for $r \leq 8$ collapse multiple PASA-like steps into single bitset operations, suggesting $O(|k|)$ cost with significantly lower constants for the hot path. The empirical data (Table 2, Section IV-E) confirms this for small- r motifs: 2.49×10^6 fused operations at 1M MSP demonstrate that $\sim 64\%$ of up-steps exploit the fast path. However, kernel management overhead (table lookups, cache misses for $r > 8$) and consolidation cost dominate at scale, preventing UPX from challenging PASA or REN. The result validates fusion as a viable optimization while confirming that neither fusion nor convolution can escape the consolidation bottleneck.

POW2BASIC. Quadratic in all regimes due to lack of structural accelerations. Serves as a baseline demonstrating that naïve application of the exponent-multiset representation (without pattern recognition or fusion) yields competitive performance only at small scales (MSP < 10k). Beyond this threshold, the accumulation of consolidation overhead without any mitigation strategy renders POW2BASIC impractical.

Unified scaling law. The decomposition in Figure 3 (Section IV-D) reveals the structural origin of quadratic runtime: step counts grow either linearly (ensembles) or quadratically (longlines) with MSP, while per-step cost grows linearly due to histogram span. Multiplying these factors yields $O(\text{MSP}^2)$ runtime in both regimes, with constants determined by algorithmic efficiency (PASA's pattern recognition, REN's streaming footprint) and input characteristics (diversity vs. uniformity). The effective slopes (Table 1, Section IV-D) cluster near 2.0 across all algorithms and regimes, confirming that no routine escapes the quadratic wall. This universality underscores the fundamental trade-off in the exponent-multiset representation: even steps are “free” (accumulated at LSP), but odd steps incur consolidation cost that scales with active exponent range. The $O(\text{MSP}^2)$ scaling is an algorithm-independent property of Collatz dynamics under P2ER, arising from the combination of step counts (linear or quadratic in MSP depending on input structure) and per-step consolidation cost (linear in MSP). The constant factors differentiating algorithms (i.e. PASA's lower overhead in diverse inputs, REN's streaming efficiency in uniform structure) are representation-induced, reflecting how each algorithm interfaces with the P2ER invariant rather than fundamental properties of the Collatz map itself.

This decomposition (as shown in Fig. 4, Table 2) clarifies why both regimes exhibit quadratic runtime despite different step count behaviors. In random ensembles, linear step growth combines with linear per-step cost to yield $O(\text{MSP}^2)$ runtime. In long waiting-lines, quadratic step growth combines with the same linear per-step cost to produce the same asymptotic scaling. The distinction lies entirely in constants (specifically: PASA's pattern recognition efficiency versus REN's streaming footprint) rather than asymptotic exponents. This universality confirms that no algorithm escapes

the consolidation bottleneck, which is fundamental to the exponent-multiset representation. However, a central lesson emerges from this hierarchy: theoretical elegance does not guarantee practical dominance.

C. A PARADOX OF ELEGANCE

A notable finding is that the most mathematically appealing algorithmic offspring (ACCEL and UPX) embody deep structural insight and theoretical advantage, yet are eclipsed in practice by simpler approaches. ACCEL recognizes that waiting-lines and gaps permit closed-form multi-step updates, compressing r elementary steps into kernel-based convolution; UPX extends this by fusing PASA-like micro-patterns into pre-computed bitset operations for small r . Both algorithms reduce iteration counts and expose structural motifs that are invisible in naïve implementations. Yet in the final hierarchy (Sections IV-B, IV-C, IV-E), neither dominates.

Specifically, PASA, with its local pattern matching, thrives on predictability and cache-friendliness. Its logic is straightforward: walk the difference sequence, emit consolidated exponents on-the-fly, but more importantly, this simplicity translates to low constants and robust performance across diverse inputs. There is no convolution, no kernel management, no fusion table lookup; just if/else branching over small integers that modern CPUs execute with near-perfect branch prediction. In random ensembles, where structural diversity prevents long runs of exploitable motifs, this simplicity is decisive.

REN, with no structural elegance and no awareness of Collatz-specific patterns, dominates adversarial longlines through representational alignment. Its bit-streaming design was not derived from exponent-space analysis but from a pragmatic engineering goal: minimize memory footprint and maximize cache locality. Yet when input structure aligns with this bias—consecutive $\Delta = 1$ motifs that map naturally to bit-level operations—REN's simplicity becomes an asset. It processes every step sequentially, but each step is cheap: no histogram construction, no carry propagation, just bitwise shifts and accumulation.

Elegance of algorithmic design does not guarantee practical supremacy. ACCEL and UPX are theoretically stronger: they compress predictable structure, reduce iteration counts, and exploit closed-form accelerations. But in both Collatz computation and broader algorithmic practice, lower constants and favorable representation can outweigh deeper structural insight. PASA's local logic and REN's streaming footprint prevail, not because they are asymptotically superior, but because they align with the realities of modern memory hierarchies and CPU microarchitectures.

The consolidation bottleneck is fundamental to the P2ER representation: maintaining strictly descending exponent vectors requires histogram-based carry propagation after every odd step, incurring $O(\text{span}(k))$ cost regardless of algorithmic sophistication. ACCEL and UPX achieve $O(n/r)$ iteration reduction, but this advantage is erased by consolidation

overhead that scales with MSP. Alternative data structures (such as lazy consolidation schemes that defer histogram updates, tree-based representations that amortize carry propagation, or approximate consolidation that tolerates temporary duplicates) could potentially reduce this per-step overhead. However, such approaches would complicate the motif detection (waiting-lines, gaps, sweeps) that currently enables PASA's pattern-based acceleration. Escaping the consolidation bottleneck without sacrificing structural awareness remains an open challenge.

D. BROADER IMPLICATIONS

The findings extend beyond Collatz computation to illuminate fundamental principles in algorithmic design for iterative numerical problems. Specifically, the conditional hierarchy established here extends beyond Collatz computation to illuminate fundamental trade-offs in algorithm design for iterative numerical problems:

No universal champion. PASA is the average-case workhorse, thriving on structural diversity where local pattern recognition minimizes redundant work. REN is the conditional specialist, excelling when input structure aligns with its bit-streaming representation. UPX and ACCEL are probes of structural acceleration, whose theoretical promise based on multi-step compression, closed-form updates, is effaced by practical overhead from consolidation and kernel management. Consequently, the hierarchy is not a ranking of algorithmic cleverness but a **map of regimes**: diversity versus uniformity, low-constant versus high-constant scaling, simplicity versus sophisticated pattern exploitation.

Representation determines performance. P2ER enables all accelerations explored here: PASA's pattern fusion, ACCEL's kernel convolution, UPX's bitset operations by making structural motifs explicit. Yet, the representation also imposes the consolidation bottleneck: histogram span grows with MSP, and binary carry propagation is unavoidable. No algorithm escapes $O(\text{MSP}^2)$ scaling in the worst case; whereas the distinction lies in constants. This underscores a broader principle: **optimal representation exposes structure, but algorithmic advantage depends on how efficiently that structure is exploited relative to the representation's inherent costs.**

Input structure governs hierarchy. The crossover between PASA and REN at 256k MSP (Section IV-C) demonstrates that algorithmic advantage is not static but conditional on input characteristics. In random ensembles, where gap jumps and short waiting-lines dominate, PASA's local consolidation outperforms REN's generic streaming. In long waiting-lines, where consecutive $\Delta = 1$ motifs saturate the input, REN's bit-level operations exploit uniformity more efficiently than PASA's pattern matching. Practitioners designing algorithms for similar iterative problems should therefore **characterize expected input distributions** and

select routines accordingly, rather than assuming existence of a single "best" algorithm.

Theoretical elegance $\neq$ practical dominance. The paradox of ACCEL and UPX, which are mathematically refined yet practically slower, echoes a broader tension in algorithm design. Asymptotic analysis ($O(n/r)$ iteration reduction) predicts advantage, but real-world performance is governed by constants, memory access patterns, and microarchitectural realities (cache hierarchies, branch prediction, TLB coverage). PASA's simplicity and REN's streaming footprint align with these realities better than ACCEL's convolution or UPX's table lookups. This suggests that **for problems dominated by memory-bound operations** (consolidation, histogram updates), **simplicity and locality often outweigh sophisticated pattern exploitation.**

Reproducibility and transparency. By providing complete benchmarking methodology (Section IV-A), reproducible input generation (random ensembles, deterministic longlines), and open discussion of verification (Section IV-E), this work establishes a standard for comparative algorithmic studies. Prior work demonstrated feasibility (Ren et al., 2018) but lacked systematic comparison or explanation of performance differences. The conditional hierarchy presented here is not anecdotal but grounded in controlled experiments, statistical validation (50-trial ensembles, $\pm 1\sigma$ confidence intervals), and reproducible artifacts. Future work on Collatz or related problems can build on this foundation, extending the hierarchy to new algorithms or testing its predictions on additional input regimes.

Generalization to other problems. The exponent-multiset representation and its attendant trade-offs (free even steps, costly consolidation, exploitable motifs) may extend to other iterative maps with multiplicative structure. Whether PASA-like pattern fusion or REN-like streaming dominates in these settings remains an open question, but the methodological framework (regime-specific benchmarking, scaling analysis, conditional hierarchy) is directly applicable. More broadly, any problem involving iterative updates to sparse numerical structures (i.e. graph algorithms on power-law networks, symbolic dynamics on numeration systems) may benefit from representation choices that make structural motifs explicit and algorithms that exploit those motifs conditionally.

VI. CONCLUSION

We have established the first systematic, large-scale benchmarking framework for Collatz progression algorithms grounded in a power-of-two exponent representation (P2ER). By executing all routines under strictly controlled single-threaded conditions, we demonstrated a reproducible conditional hierarchy across random ensembles and deterministic long waiting-lines.

The results are quantitatively definitive: PASA dominates average-case ensembles up to 500k MSP, maintaining 5-10% lower runtime through on-the-fly consolidation. At the

256k MSP crossover, REN's bit-streaming alignment with consecutive waiting-line motifs becomes decisive: by 1M MSP, REN completes in 7.3 hours versus PASA's 9.4 hours (30% faster). UPX validates fusion strategies—executing 2.49M fused operations at 1M MSP—but consolidation overhead prevents it from challenging PASA or REN. This confirms the central finding: algorithmic advantage is regime-dependent, not universal. PASA excels on structural diversity, REN on uniform motifs, neither dominates unconditionally.

Scaling analysis reveals the unified law: both regimes exhibit quadratic runtime $O(\text{MSP}^2)$, with differences arising from constants rather than exponents. PASA maintains lower constants in ensembles through pattern recognition; REN reduces constants in longlines through streaming efficiency. Neither escapes the consolidation bottleneck fundamental to the exponent-multiset representation.

Record computations extend the frontier from $2^{\{100,000\}-1}$ to $2^{\{1,024,000\}-1}$, which is a 900,000-fold MSP increase representing $\sim 10^{\{278,000\}}$ in absolute magnitude, thus establishing the longest Collatz progression ever computed (13.8M steps). Rigorous verification confirms correctness at this unprecedented scale, demonstrating that the conditional hierarchy persists across three orders of magnitude.

The broader lesson: elegance of algorithmic design does not guarantee practical supremacy. ACCEL and UPX embody sophisticated structural insight such as kernel compression, closed-form acceleration, yet are eclipsed by PASA's humble local logic and REN's streaming footprint. Lower constants and favorable representation prevail where theoretical refinement falters.

In summary, this work transforms Collatz computation from anecdotal engineering into systematic comparative science. By exposing invariants that govern efficiency, establishing reproducible benchmarks at unprecedented scale, and demonstrating that performance is conditional on input structure, we provide both a definitive hierarchy and a methodological template for future algorithmic studies. P2ER and regime-specific analysis framework generalize to other iterative maps, symbolic dynamics on numeration systems—wherever multiplicative structure admits similar decomposition. The performance hierarchy established here reflects single-threaded CPU execution. Unlike breadth-first verification, which is embarrassingly parallel, single-trajectory computation is inherently sequential: each step depends on the previous state, offering no natural parallelism for GPU exploitation. Experiments with GPU-accelerated kernel convolutions showed no advantage over optimized CPU implementations. The hierarchy reported here is therefore likely robust across architectures for single-trajectory computation.

Several directions merit future investigation: (1) Lazy consolidation strategies: Deferring histogram updates across

multiple steps could amortize the per-step consolidation cost that currently dominates runtime.

(2) Extension to generalized maps: The P2ER framework and regime-conditional analysis generalize naturally to $ax+b$ iterative maps with multiplicative structure, enabling systematic study of related dynamical systems.

(3) Machine learning for algorithm selection: Given input structure statistics (motif frequencies, gap distributions), predictive models could automatically select optimal algorithms, achieving best-of-both-worlds performance across regimes.

(4) Theoretical analysis: Connecting empirical scaling laws to provable complexity bounds under structural assumptions remains an open challenge.

The Collatz conjecture remains unproven, but its algorithmic landscape is no longer anecdotal. It is mapped, measured, and explained. The P2ER exposes structural accelerations (waiting-lines, gaps, carry cascades) that emerge naturally from mathematics. Whether this framework contributes to eventual proof remains unknown. What is clear is that it provides new tools for systematic exploration and has extended computational verification by three orders of magnitude.

CODE AND DATA AVAILABILITY

Source code for all five algorithmic families (PASA, REN, UPX, ACCEL, POW2BASIC), the benchmarking framework, and complete experimental datasets (random ensembles and deterministic waiting-line inputs) are available from the corresponding author upon reasonable request (nalic@ucsd.edu). A permanent institutional repository will be established upon acceptance, with the URL to be provided in the final published version.

REFERENCES

- [1] J.-P. Allouche, "Sur la conjecture de 'Syracuse-Kakutani-Collatz,'" Séminaire de Théorie des Nombres, CNRS, Talence, France, Tech. Rep. 9, 1979.
- [2] J. C. Lagarias, "The $3x + 1$ problem and its generalizations," *Amer. Math. Monthly*, vol. 92, no. 1, pp. 3–23, Jan. 1985, doi: [10.2307/2322189](https://doi.org/10.2307/2322189).
- [3] T. Oliveira e Silva, "Empirical verification of the $3x+1$ and related conjectures," in *The Ultimate Challenge: The $3x+1$ Problem*, J. C. Lagarias, Ed., Providence, RI, USA: American Mathematical Society, 2010, pp. 189–207. [Online]. Available: <http://sweet.ua.pt/tos/3x+1.html>
- [4] W. Ren, S. Li, R. Xiao, and W. Bi, "Collatz conjecture for $2^{1000000}-1$ is true-Algorithms for verifying extremely large numbers," in *Proc. IEEE SmartWorld/UIC/ATC/ScalCom/CBDCom/ IoP/SCI*, Guangzhou, China, Oct. 2018, pp. 411–418, doi: [10.1109/SmartWorld.2018.00099](https://doi.org/10.1109/SmartWorld.2018.00099).
- [5] E. S. Getachew and B. G. Assefa, "Efficient computation of collatz sequence stopping times: A novel algorithmic approach," *IEEE Access*, vol. 13, pp. 41210–41220, 2025.
- [6] D. Barina, "Improved verification limit for the convergence of the collatz conjecture," *J. Supercomput.*, vol. 81, no. 7, pp. 1–14, May 2025, doi: [10.1007/s11227-025-07337-0](https://doi.org/10.1007/s11227-025-07337-0).
- [7] I. Korec, "A density estimate for the $3x+1$ problem," *Math. Slovaca*, vol. 44, no. 1, pp. 85–89, 1994, doi: [10.1515/ms-1994-0109](https://doi.org/10.1515/ms-1994-0109).
- [8] I. Krasikov and J. C. Lagarias, "Bounds for the $3x+1$ problem using difference inequalities," *Acta Arithmetica*, vol. 109, no. 3, pp. 237–258, 2003, doi: [10.4064/aa109-3-4](https://doi.org/10.4064/aa109-3-4).

- [9] T. Tao, "Almost all orbits of the Collatz map attain almost bounded values," *Forum Math., Pi*, vol. 10, pp. 1–56, Jan. 2022, doi: [10.1017/fmp.2022.8](https://doi.org/10.1017/fmp.2022.8).
- [10] M. D. Ercegovic and T. Lang, *Digital Arithmetic*. San Mateo, CA, USA: Morgan Kaufmann, 2004.
- [11] J. P. More. *Fundamental Equation of Collatz Conjecture*. Accessed: Apr. 10, 2026. [Online]. Available: https://youtu.be/Zdt2xU_1geU



NIKOLA ALIC (Member, IEEE) received the B.Sc. degree in optoelectronics and laser engineering from Belgrade University, Serbia, in 1998, and the M.Sc. and Ph.D. degrees in photonics from the University of California, San Diego (UCSD), CA, USA, in 2001 and 2006, respectively.

He is currently the Director of the Photonics Laboratory and Circuits Laboratory at the Qualcomm Institute, UCSD, and holds the position of

an Associate Research Scientist with California Institute for Telecommunications and Information Technologies. His pioneering work introduced electronic equalization principles that enabled modern high-speed transmission links, including the first experimental demonstrations of maximum-likelihood sequence estimation (MLSE) at 10 Gb/s and long-haul propagation without in-line optical dispersion compensation. He subsequently solved the problem of nonlinear Kerr effect cancellation in wavelength-division multiplexed (WDM) systems through frequency-referenced carrier techniques, demonstrating transmission beyond the nonlinear capacity limit in optical fibers (Science, 2015).

Dr. Alic was the Program Chair, in 2017, and the General Chair, in 2019, of the IEEE Photonics Society Annual Meeting, the General Co-Chair of Frontiers in Optics, in 2015, and has served on the Technical Program Committees of numerous conferences including OFC, where he held the position of the Sub-Committee Chair, in 2012. He was an Associate Editor for Optics Express, in 2011 to 2016. He served as an Associate Editor, from 2012 to 2017, and the Deputy Editor, from 2018 to 2022, for the IEEE/OSA Journal of Lightwave Technology and continues as a member of its Steering Committee.

...